

# Real Estate Investment Trusts Stock Analysis

```
In [1]: # Importing necessary libraries
```

```
import numpy as np
```

```
In [2]: # Loading the dataset for SBRA (Sabra Health Care REIT, Inc. data sample)
```

```
adjusted_closings_sbra = np.loadtxt('SBRA.csv', skiprows=1, usecols=5, delimiter=',')
print(adjusted_closings_sbra)
```

```
[15.36233 16.012386 16.528606 16.882313 17.427212 17.522808 17.513248
17.427212 17.264698 17.417652 17.847836 18.105947 18.344938 18.306698
18.497892 18.77512 19.09059 19.386936 19.377378 19.55901 19.635489
19.425177 19.664167 19.539892 19.635489 19.80756 19.654606 19.60681
19.358257 19.32958 19.339357 19.544678 19.388243 19.241585 19.222029
19.143812 18.635395 18.381187 18.038986 17.716337 17.31547 17.247028
17.256807 17.149258 17.119926 17.071039 17.383911 17.608788 17.941214
17.970543 17.892326 17.726112 17.765223 17.980322 18.58651 18.293192
18.449629 18.762501 18.870049 19.192698 19.036261 18.899382 18.879826
18.694059 18.762501 18.958044 18.870049 18.801609 19.04604 19.241585
19.241585 19.222029 18.821163 18.361633 18.713614 18.410519 18.713614
18.909159 18.909159 19.153589 18.967821 19.124256 19.094927 19.143812
19.554455 19.456682 19.143812 19.319801 18.840717 19.310024 19.241585
19.417574 19.574011 19.75 19.709999 19.530001 19.73 19.700001
19.709999 19.75 19.540001 19.219999 19.370001 19.290001 19.24
19.1 19.32 19.450001 19.49 19.32 19.290001 19.32
19.48 19.43 19.790001 19.58 20.139999 20.049999 19.57
19.309999 19.26 18.870001 19.450001 19.690001]
```

```
In [3]: # Loading the dataset for EQR (Equity Residential data sample)
```

```
adjusted_closings_eqr = np.loadtxt('eqr.csv', skiprows=1, usecols=5, delimiter=',')
print(adjusted_closings_eqr)
```

```
[62.800674 63.19466 63.588642 64.120514 65.322166 65.322166 66.395767
66.533653 66.61245 67.429955 68.040634 68.858147 69.064987 69.340775
69.28167 69.291527 69.606705 70.640915 71.281136 70.916695 71.468269
70.936401 71.694817 71.852402 71.901649 72.433533 72.335037 72.354729
71.448578 72.187286 72.019852 72.10849 72.423676 71.586472 72.748718
73.113152 72.394127 72.610817 72.226692 72.581276 72.394127 72.866913
72.817657 72.443382 72.325188 72.532021 73.319992 73.920807 73.782921
73.930656 73.635178 72.866913 72.394127 72.512329 73.664719 73.841385
73.910858 74.575829 74.41703 75.062149 74.754478 74.41703 75.121704
74.823952 74.714775 75.042297 74.992676 74.823952 75.776741 76.114189
76.868484 77.017357 75.1614 73.732208 74.506355 73.613106 74.397179
74.903351 75.399597 75.627869 74.734627 75.846214 75.945465 75.82637
76.014938 75.637794 74.208603 73.881088 74.258232 75.260651 75.270569
75.240791 75.61795 76.233284 76.044716 75.181252 75.786667 76.064568
76.531036 76.610435 75.637794 75.101845 75.250725 75.995094 75.667564
74.913277 76.669983 77.205933 77.394508 77.245636 77.215858 77.066986
77.344879 77.672401 78.079323 77.463982 77.841125 78.238121 77.44413
77.682327 76.332542 75.082001 75.610001 75.919998]
```

## Simple Rate of Return function

Using the adjusted closing price, we can calculate the daily rate of return. The formula to be used is  $out[n] = a[n + 1] - a[n]$  (formula: `np.diff()`).

```
In [4]: # Defining the function to get the simple rate of return
```

```
def rate_of_return(adj_closings):
    daily_simple_ror = np.diff(adj_closings)/adj_closings[:-1]
    return daily_simple_ror
```

```
In [5]: # Calculating the Simple Rate of Return for SBRA
```

```
daily_simple_returns_sbra = rate_of_return(adjusted_closings_sbra)
print(daily_simple_returns_sbra)
```

```
[ 0.04231494  0.03223879  0.02139969  0.03227632  0.00548544 -0.00054557
-0.00491262 -0.0093253  0.00885935  0.02469816  0.01446175  0.01319959
-0.0020845  0.01044394  0.01498701  0.01680256  0.01552315 -0.00049301
 0.0093734  0.00391017 -0.01071081  0.01230311 -0.00631987  0.0048924
 0.00876327 -0.007722  -0.0024318  -0.01267687 -0.00148138  0.00050581
 0.01061674 -0.00800397 -0.00756427 -0.00101634 -0.00406913 -0.02655777
-0.01364114 -0.01861692 -0.0178862  -0.02262697 -0.00395265  0.000567
-0.00623227 -0.00171039 -0.00285556  0.01832765  0.01293593  0.01887841
 0.00163473 -0.00435251 -0.00928968  0.00220641  0.01210787  0.03371397
-0.01578123  0.00855165  0.01695817  0.00573207  0.01709847 -0.00815086
-0.00719044 -0.00103474 -0.00983944  0.00366116  0.01042201 -0.00464157
-0.00362691  0.01300054  0.01026696  0.          -0.00101634 -0.02085451
-0.0244156  0.01916937 -0.0161965  0.01646314  0.01044934  0.
 0.01292654 -0.00969886  0.00824739 -0.0015336  0.0025601  0.02145043
-0.00500004 -0.01608034  0.009193  -0.02479756  0.02490919 -0.00354422
 0.00914628  0.00805646  0.00899095 -0.00202537 -0.00913232  0.0102406
-0.00152048  0.00050751  0.00202948 -0.01063286 -0.01637676  0.00780447
-0.0041301  -0.00259207 -0.00727651  0.01151832  0.00672883  0.0020565
-0.00872242 -0.00155274  0.00155516  0.00828157 -0.00256674  0.0185281
-0.01061147  0.02860056 -0.00446872 -0.0239401  -0.01328569 -0.00258928
-0.02024917  0.03073662  0.01233933]
```

```
In [6]: # Calculating the Simple Rate of Return for EQR
```

```
daily_simple_returns_eqr = rate_of_return(adjusted_closings_eqr)
print(daily_simple_returns_eqr)
```

```
[ 0.0062736  0.00623442  0.00836426  0.01874052  0.          0.01643548
 0.00207673  0.00118432  0.01227256  0.00905649  0.01201507  0.00300386
 0.00399317 -0.00085238  0.00014227  0.00454858  0.01485791  0.00906303
-0.00511273  0.00777777 -0.00744202  0.01069149  0.002198  0.00068539
 0.00739738 -0.00135981  0.00027223 -0.01252373  0.01033902 -0.00231944
 0.00123074  0.004371  -0.01155981  0.01623555  0.00500949 -0.00983441
 0.0029932  -0.00529019  0.00490932 -0.00257847  0.00653072 -0.00067597
-0.00513989 -0.00163154  0.00285976  0.01086377  0.00819442 -0.00186532
 0.00200229 -0.00399669 -0.0104334  -0.00648835  0.00163276  0.01589233
 0.00239824  0.00094084  0.00899693 -0.00212936  0.00866897 -0.00409888
-0.00451408  0.00946926 -0.00396359 -0.00145912  0.00438363 -0.00066124
-0.00224987  0.01273374  0.00445319  0.00991004  0.00193672 -0.02409791
-0.01901497  0.01049944 -0.0119889  0.01065127  0.00680365  0.00662515
 0.0030275  -0.01181102  0.01487379  0.00130858 -0.00156816  0.00248684
-0.00496145 -0.0188952  -0.00441344  0.00510474  0.0134991  0.00013178
-0.00039561  0.00501269  0.00813741 -0.00247357 -0.01135469  0.00805274
 0.00366689  0.00613253  0.00103747 -0.01269593 -0.00708573  0.00198237
 0.00989185 -0.00430988 -0.00996843  0.02344986  0.00699035  0.00244249
-0.00192355 -0.0003855  -0.001928  0.00360586  0.00423457  0.00523895
-0.00788097  0.00486862  0.00510008 -0.01014839  0.00307573 -0.0173757
-0.0163828  0.00703231  0.00409995]
```

## Calculating the Average Daily Returns

```
In [7]: average_daily_simple_return_sbra = np.mean(daily_simple_returns_sbra)
print(average_daily_simple_return_sbra)
```

```
0.00210937212011956
```

```
In [8]: average_daily_simple_return_eqr = np.mean(daily_simple_returns_eqr)
print(average_daily_simple_return_eqr)
```

```
0.0015777637451981398
```

## Calculating the Annulized Daily Log Return

```
In [9]: # Defining a function that returns the annualized log return
```

```
def annualize_log_return(daily_log_returns):  
    average_daily_log_returns = np.mean(daily_log_returns)  
    annualized_log_return = average_daily_log_returns * 250  
    return annualized_log_return
```

```
In [10]: # Annualizing the log return of SBRA
```

```
annualized_log_return_sbra = annualize_log_return(daily_simple_returns_sbra)  
print(annualized_log_return_sbra)
```

```
0.52734303002989
```

```
In [11]: # Annualizing the log return of EQR
```

```
annualized_log_return_eqr = annualize_log_return(daily_simple_returns_eqr)  
print(annualized_log_return_eqr)
```

```
0.39444093629953497
```

## Calculating the Variance of Daily Log Returns

### Calculating the variance for SBRA

```
daily_variance_sbra = np.var(daily_simple_returns_sbra) print(daily_variance_sbra)
```

```
In [13]: # Calculating the variance for EQR
```

```
daily_variance_eqr = np.var(daily_simple_returns_eqr)  
print(daily_variance_eqr)
```

```
6.83056405613e-05
```

## Calculating the Standard Deviation of the Daily Log Returns

```
In [14]: # Calculating the standard deviation for SBRA
```

```
daily_sd_sbra = np.std(daily_simple_returns_sbra)  
print(daily_sd_sbra)
```

```
0.013410384045669034
```

```
In [16]: # Calculating the standard deviation for EQR
```

```
daily_sd_eqr = np.std(daily_simple_returns_eqr)  
print(daily_sd_eqr)
```

```
0.008264722654832406
```

## Calculating the Correlation between SBRA and EQR

```
In [17]: corr_sbra_eqr = np.corrcoef(daily_simple_returns_sbra, daily_simple_returns_eqr)  
print(corr_sbra_eqr)
```

```
[[1.          0.61941465]  
 [0.61941465 1.          ]]
```

Both stocks are moderately correlated, which means they should respond similarly to external economic forces.

## Conclusion

For a slightly more stable investment, I would recommend investing in EQR. However, if higher returns are sought (and higher risk is tolerated), then investing in SBRA would be ideal.

